

# Fact-based News Verification System

A Hybrid System with BERT Classification, GNews Evidence Retrieval, and  
Fine-tuned NLI Verification

| Name         | Student ID | Email              |
|--------------|------------|--------------------|
| Sun Yuchen   | A0326248B  | e1538079@u.nus.edu |
| Zhao Jiahui  | A0329852U  | e1554179@u.nus.edu |
| Zhang Yuxuan | A0285664N  | e1216649@u.nus.edu |

**Abstract:** This project develops a fact-based news verification system for online news articles. Instead of relying only on article-level fake/real classification, the system combines a fine-tuned BERT baseline classifier with sentence-level evidence retrieval and a fine-tuned DeBERTa-based Natural Language Inference verifier. The input article is cleaned and split into sentence-level verification units. Each sentence is used to retrieve external evidence from GNews, and the NLI model determines whether the evidence supports, refutes, or provides insufficient information for the sentence. The sentence-level results are aggregated into an article-level verdict. The system also includes sentiment analysis, LDA-based topic analysis, and a Streamlit chat interface to improve interpretability and user interaction.

**Keywords:** BERT Classification, GNews Retrieval, Natural Language Inference, Evidence Verification, Sentiment Analysis, Topic Modelling, Streamlit Chat

## Contents

|          |                                                     |          |
|----------|-----------------------------------------------------|----------|
| <b>1</b> | <b>Business Problem</b>                             | <b>1</b> |
| <b>2</b> | <b>Datasets Used</b>                                | <b>1</b> |
| 2.1      | Baseline Fake/Real Classification Dataset . . . . . | 1        |
| 2.2      | NLI Training Dataset . . . . .                      | 2        |
| 2.3      | Auxiliary Claim Extraction Dataset . . . . .        | 3        |
| 2.4      | Live Evidence Source . . . . .                      | 3        |
| 2.5      | Topic Analysis Dataset . . . . .                    | 3        |
| <b>3</b> | <b>Solution Approach</b>                            | <b>3</b> |
| 3.1      | System Overview . . . . .                           | 3        |
| 3.2      | Text Cleaning . . . . .                             | 4        |
| 3.3      | BERT Baseline Classifier . . . . .                  | 4        |
| 3.4      | Sentence-level GNews Evidence Retrieval . . . . .   | 4        |
| 3.5      | Fine-tuned DeBERTa-NLI Verification . . . . .       | 4        |
| 3.6      | Article-level Aggregation . . . . .                 | 5        |
| 3.7      | Sentiment Analysis . . . . .                        | 5        |
| 3.8      | LDA Topic Analysis . . . . .                        | 5        |
| 3.9      | Chat Interface . . . . .                            | 5        |
| 3.10     | Implementation Modules and Configuration . . . . .  | 6        |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| <b>4</b> | <b>Test Results</b>                          | <b>6</b>  |
| 4.1      | BERT Baseline Evaluation . . . . .           | 6         |
| 4.2      | NLI Verifier Evaluation . . . . .            | 7         |
| 4.3      | Model Comparison Summary . . . . .           | 8         |
| 4.4      | End-to-end Verification Case Study . . . . . | 8         |
| 4.5      | Error Analysis and Discussion . . . . .      | 9         |
| <b>5</b> | <b>Limitations and Future Work</b>           | <b>9</b>  |
| <b>6</b> | <b>Conclusion</b>                            | <b>10</b> |
| <b>7</b> | <b>References</b>                            | <b>10</b> |
| 7.1      | Libraries and Frameworks . . . . .           | 10        |
| 7.2      | Models, APIs, and Data Sources . . . . .     | 10        |
| <b>8</b> | <b>Individual Contributions</b>              | <b>11</b> |
| <b>9</b> | <b>GenAI / LLM Declaration</b>               | <b>11</b> |

# 1 Business Problem

Online news spreads quickly, but ordinary readers rarely have time to check whether the factual statements in an article are actually supported by external evidence. This creates a practical verification problem rather than only a text classification problem. A news story may look professional, use neutral tone, and resemble real reporting style while still containing unsupported, outdated, or false factual statements.

This motivates a shift away from treating the task as simple fake/real classification. Three problems are especially important:

- article-level fake/real classifiers cannot explain which sentence or claim is false;
- style-based classifiers may learn topic bias, source bias, or writing patterns instead of factual correctness;
- real-world articles often mix true, false, and unverifiable statements in the same document.

The goal of this project is therefore to build an explainable first-pass verification tool that retrieves external evidence and verifies sentence-level factual statements. The system is designed to:

- classify the overall article using a BERT baseline model;
- split the article into sentence-level verification units;
- retrieve external evidence from GNews for each sentence;
- use a fine-tuned NLI model to classify evidence-sentence pairs as supported, refuted, or not enough information (NEI);
- aggregate sentence-level results into an article-level verdict;
- provide sentiment, topic, and chat-based interaction features for interpretability and usability.

The intended users are students, researchers, journalists, and general readers who want a transparent way to inspect whether a news article is supported by other reporting. The system is not a replacement for professional fact-checking. Instead, it serves as an explainable verification assistant that combines fast screening with evidence-based semantic reasoning.

## 2 Datasets Used

The final report narrative separates the data sources by their role in the pipeline rather than treating everything as one generic dataset. The system uses local fake/real datasets for baseline classification, a custom news NLI dataset for verifier fine-tuning, a small claim-extraction training set for an auxiliary extractor module, GNews as a live evidence source at inference time, and the fake/real datasets again for topic exploration.

### 2.1 Baseline Fake/Real Classification Dataset

The local repository uses four FakeNewsNet-style CSV files for the baseline BERT classifier. These files mainly provide title-level text rather than full article bodies, so the baseline model should be described honestly as a title or headline classifier with limited article context.

These files are suitable for a baseline classifier because they provide labeled examples of fake and real news headlines. However, because they do not mainly contain full article bodies, they are insufficient for sentence-level factual verification on their own.

The codebase also contains LIAR-style TSV loading support for `train.tsv`, `valid.tsv`, and `test.tsv`. However, the final reported baseline model is the saved BERT classifier trained on the local GossipCop and PolitiFact split, because that is the stable pipeline used in the final implementation.

Table 1: Local fake/real datasets used for baseline classification

| File                             | Rows          | Label | Role                                                      |
|----------------------------------|---------------|-------|-----------------------------------------------------------|
| <code>gossipcop_fake.csv</code>  | 5,323         | Fake  | Title-level fake-news examples for BERT training.         |
| <code>gossipcop_real.csv</code>  | 16,817        | Real  | Title-level real-news examples for BERT training.         |
| <code>politifact_fake.csv</code> | 432           | Fake  | Additional fake-news titles for domain diversity.         |
| <code>politifact_real.csv</code> | 624           | Real  | Additional real-news titles for domain diversity.         |
| <b>Total</b>                     | <b>23,196</b> | –     | Local baseline data with 17,441 real and 5,755 fake rows. |

## 2.2 NLI Training Dataset

For the NLI verifier, the project uses a custom news-related NLI dataset stored under `data/nli_dataset/`. It contains 39,000 balanced three-way samples: 30,000 for training, 4,500 for validation, and 4,500 for testing. Each example pairs a news evidence passage with a claim-like hypothesis, which makes the data much closer to the deployed verification task than binary fake/real classification. The fine-tuning workflow is implemented in `scripts/train_nli_verifier.py`.

Table 2: Custom NLI dataset used for verifier fine-tuning

| Split                             | Samples       | Class Balance           | Role                                          |
|-----------------------------------|---------------|-------------------------|-----------------------------------------------|
| <code>real_nli_train.jsonl</code> | 30,000        | Balanced 3-way          | Fine-tuning the DeBERTa verifier.             |
| <code>real_nli_valid.jsonl</code> | 4,500         | Balanced 3-way          | Model selection and threshold tuning.         |
| <code>real_nli_test.jsonl</code>  | 4,500         | Balanced 3-way          | Final held-out evaluation.                    |
| <b>Total</b>                      | <b>39,000</b> | <b>13,000 per class</b> | Custom news NLI corpus for fact verification. |

Table 3: Sample fields in the NLI dataset

| Field                   | Meaning                                               |
|-------------------------|-------------------------------------------------------|
| <code>premise</code>    | Evidence text or retrieved news passage.              |
| <code>hypothesis</code> | Sentence or factual statement to be verified.         |
| <code>label_id</code>   | NLI target: 0=contradiction, 1=neutral, 2=entailment. |

At inference time, these NLI outputs are mapped into verification labels as follows:

- contradiction → **refuted**;
- neutral → **NEI**;
- entailment → **supported**.

This custom dataset is the key training resource for the fine-tuned verifier and must be documented explicitly, because it is one of the main differences between the final implementation and an earlier heuristic prototype.

## 2.3 Auxiliary Claim Extraction Dataset

The repository also includes a small custom training set for the claim extractor, stored as `data/claim_extraction_dataset.json`. This dataset is used to fine-tune a Flan-T5 sequence-to-sequence model that rewrites longer article text into claim-like verification units. The training workflow is implemented in `scripts/train_claim_extractor.py`.

In the current application, however, sentence splitting is the default retrieval strategy because preserving the original sentence wording produced more reliable GNews results than aggressively compressed claims. The claim extractor remains a documented auxiliary component with two modes: a learned seq2seq extraction mode and a regex-based rule fallback.

## 2.4 Live Evidence Source

GNews is not treated as a training dataset. Instead, it is used as a live external evidence source at inference time. For each sentence-level verification unit, the system queries GNews and collects returned article metadata such as title, description, content snippet, source, publication date, and URL.

This distinction matters for the report narrative: the baseline classifier and NLI verifier are trained offline, while GNews provides current evidence during runtime. The retrieval step therefore connects the static models with up-to-date news reporting.

## 2.5 Topic Analysis Dataset

The same local GossipCop and PolitiFact data are also used for topic analysis. In this module, the labels are not used for verification decisions. Instead, the documents are vectorized to explore recurring themes, topic-word distributions, fake/real topic patterns, and article-topic assignments in the training corpus.

# 3 Solution Approach

The final system should be described as an evidence-based news verification pipeline rather than as a simple fake-news classifier. The baseline BERT model remains important, but it now serves as an article-level prior alongside a sentence-level retrieval and NLI verification pipeline.

## 3.1 System Overview

The end-to-end flow of the system is shown below:

```
Article Input → Text Cleaning → BERT Baseline Classifier
→ Sentence Splitting → GNews Retrieval → Evidence Reranking
→ Fine-tuned DeBERTa-NLI Verifier → Article-level Aggregation
→ Final Verdict + Evidence + Sentiment + Topic + Chat
```

This architecture balances speed and interpretability. The BERT classifier gives an immediate article-level signal, while the retrieval and NLI stages explain which specific factual statements are supported, contradicted, or still unverified.

The implementation also includes saved model artefacts for the main learned components:

- `models/baseline_classifier/final/` for the baseline BERT classifier;
- `models/claim_verifier/final/` and intermediate checkpoints under `models/claim_verifier/`;
- `models/claim_extractor/final/` for the trained Flan-T5 claim extractor.

### 3.2 Text Cleaning

Before classification and retrieval, the input article is normalized to reduce noise. The cleaning stage removes URLs, strips social-media artefacts, and normalizes whitespace and punctuation. This improves consistency for both the baseline classifier and the retrieval module.

The cleaning step is deliberately lightweight. Its goal is not to rewrite the article, but to preserve the original wording as much as possible while making the input safe and stable for downstream processing.

### 3.3 BERT Baseline Classifier

The baseline model uses `bert-base-uncased` fine-tuned for binary fake/real classification. Because the available local data are mainly title-level records, the classifier should be presented as a fast article-level prior rather than as a complete factual verifier.

This distinction is important for the report. The BERT classifier does not determine whether the article is true or false in a strict fact-checking sense. Instead, it estimates whether the article resembles fake or real examples in the training data. Its output is later combined with evidence-based verification signals rather than used as the only decision criterion.

### 3.4 Sentence-level GNews Evidence Retrieval

The codebase contains both a trained claim extractor and a simpler sentence splitter. In practice, the current Streamlit application defaults to sentence splitting through retrieval-unit segmentation, because earlier experiments with aggressive claim simplification often reduced search quality. Important context was sometimes lost, making the query too vague or too distorted for reliable news search.

Each sentence is preserved as much as possible and submitted to GNews after query-safe sanitization. The retrieval logic focuses on:

- punctuation removal and cleanup that do not change the core wording;
- query length control for API-safe search strings;
- preserving original word order whenever possible;
- searching over returned title, description, and content fields;
- reranking evidence based on lexical overlap, entities, numbers, and source relevance;
- source credibility scoring, where manually rated outlets are grouped into tiers and integrated into the ranking formula.

This design improves explainability because the system can show users exactly which sentence triggered which retrieval query and which evidence was selected for verification. It also reflects a practical design decision: the retriever does not trust all sources equally, so wire services and highly reputable news organizations contribute more strongly to the final evidence score than weak or noisy outlets.

### 3.5 Fine-tuned DeBERTa-NLI Verification

The central upgrade in the final system is the move from a development-stage heuristic overlap matcher to a fine-tuned Natural Language Inference verifier. The deployed verifier is initialized from `MoritzLaurer/deberta-v3-base-mnli-fever-anli` and then fine-tuned on the custom 39K-sample news NLI dataset described in Section 2.2. The NLI model receives the retrieved evidence as the premise and the article sentence as the hypothesis:

- **Premise:** retrieved evidence snippet;

- **Hypothesis:** sentence from the input article.

The NLI output is mapped into verification labels as follows:

- entailment → supported;
- contradiction → refuted;
- neutral → NEI.

Long retrieved evidence is processed in chunks rather than as one undifferentiated block. Each claim-evidence chunk is scored separately, echo or parroting effects are discounted, and the strongest NLI signal is selected instead of averaging across irrelevant passages. This design is more robust than heuristic token matching because it models contradiction, paraphrase, and relevance at the sentence-pair level while reducing factual dilution from noisy snippets.

### 3.6 Article-level Aggregation

The system aggregates sentence-level verification outputs into a final article-level verdict. A practical aggregation rule is:

- many supported sentences → **Likely True**;
- many refuted sentences → **Likely False**;
- mostly NEI results or weak evidence → **Unverified**.

The BERT baseline is used as an additional article-level signal rather than as a hard override. This hybrid design allows the final verdict to reflect both the training-data prior and the retrieved evidence trail.

### 3.7 Sentiment Analysis

Sentiment analysis is included as an interpretability feature rather than a truth-verification signal. The primary model is `cardiffnlp/twitter-roberta-base-sentiment-latest`. If that model is not available, the pipeline falls back first to NLTK VADER and then to a simple keyword-based scorer. The output labels are positive, neutral, and negative.

This module helps users understand the tone of the submitted article or the tone of retrieved reporting. However, sentiment does not determine whether a story is true or false. A false article can sound neutral, and a true article can be emotionally framed, so sentiment is used only to improve interpretability.

### 3.8 LDA Topic Analysis

The topic analysis component uses `CountVectorizer` and Latent Dirichlet Allocation (LDA) as the always-available topic model, with optional `BERTopic` support when the required dependencies are available. The topic page can show top words per topic, sample documents, document-topic probabilities, article topic inference, heatmaps, and fake/real distributions by topic.

This analysis is useful for exploratory understanding of the dataset and for showing how different topics appear in fake and real subsets. It is not part of the final truth-verification decision and should be described as an interpretability and exploratory-analysis feature.

### 3.9 Chat Interface

The Streamlit chat tab provides a conversational interface over the system’s functions. Through chat-style prompts, the user can load an article, trigger verification, inspect sentence-level search units, view sentiment output, obtain short article previews, and ask keyword-triggered follow-up questions.

To keep the report accurate, the chat interface should be described honestly as a rule-based interaction layer built with Streamlit chat components. It maps user keywords or menu choices to existing verification functions, and it caches pipeline results so that repeated interactions do not recompute the full workflow unnecessarily. It is not an open-ended large-language-model chatbot.

### 3.10 Implementation Modules and Configuration

The final system includes several modules beyond the core BERT and NLI models. Many runtime thresholds and scoring weights are centralized in `pyproject.toml` and can also be overridden through environment variables, which makes the verification pipeline easier to tune without rewriting code.

Table 4: Main implementation modules and configuration files

| Module or File                           | Responsibility                                                                                                          |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| <code>app.py</code>                      | Streamlit interface, multi-tab workflow, chat interaction, and cached end-to-end orchestration.                         |
| <code>src/retriever.py</code>            | GNews querying, retrieval-unit search, source credibility scoring, deduplication, and evidence reranking.               |
| <code>src/verifier.py</code>             | Fine-tuned DeBERTa NLI verification with chunked evidence processing, echo discounting, and strongest-signal selection. |
| <code>src/claim_extractor.py</code>      | Flan-T5 claim extraction with a rule-based fallback; retained as an auxiliary module.                                   |
| <code>src/sentiment_analyser.py</code>   | Media-tone analysis using RoBERTa sentiment with VADER and keyword fallbacks.                                           |
| <code>src/topic_analysis_page.py</code>  | LDA topic exploration, optional BERTopic integration, dataset explorer, and article-topic inference.                    |
| <code>models/baseline_classifier/</code> | Saved BERT baseline classifier artefacts used for article-level screening.                                              |
| <code>models/claim_verifier/</code>      | Saved fine-tuned DeBERTa verifier artefacts and intermediate checkpoints.                                               |
| <code>pyproject.toml</code>              | Central configuration for retrieval weights, verifier thresholds, and other overridable system parameters.              |

## 4 Test Results

The final implementation includes saved trained models and recorded evaluation results, so this section reports actual numbers rather than placeholder tables. The trained artefacts are saved under the baseline classifier, claim verifier, and claim extractor model directories described in Section 3.10.

### 4.1 BERT Baseline Evaluation

The baseline BERT classifier was evaluated on a 20% held-out test split with 4,640 samples. The results confirm that the baseline model is a strong article-level screening signal, but they also reveal the expected class-imbalance effect between fake and real news.

The fine-tuned BERT baseline achieves accuracy 0.8752, binary F1 0.9189, and macro-F1 0.8245 on the held-out split. At the class level, fake-news precision, recall, and F1 are 0.7877, 0.6803, and 0.7301 respectively, while the corresponding real-news values are 0.8991, 0.9395, and 0.9189. These



Table 5: Confusion matrix for the BERT baseline classifier

| <b>Actual / Predicted</b> | <b>Pred Fake</b> | <b>Pred Real</b> |
|---------------------------|------------------|------------------|
| Actual Fake               | 783              | 368              |
| Actual Real               | 211              | 3,278            |

numbers show clear class-imbalance effects: approximately 32% of fake news in the test split is misclassified as real. This is consistent with the roughly 3:1 real-to-fake ratio in the training data and reinforces the decision to treat the BERT model as a prior rather than as the sole truth-verification mechanism.

To make the effect of fine-tuning explicit, Table 6 compares the final fine-tuned classifier with two weaker baselines: a majority classifier that always predicts **Real**, and the original **bert-base-uncased** model with an untrained randomly initialized classification head.

Table 6: Baseline classifier comparison on the 4,640-sample held-out test set

| <b>Model</b>                    | <b>Accuracy</b> | <b>P<br/>(binary)</b> | <b>R<br/>(binary)</b> | <b>F1<br/>(binary)</b> | <b>F1<br/>(macro)</b> |
|---------------------------------|-----------------|-----------------------|-----------------------|------------------------|-----------------------|
| Majority baseline (always Real) | 0.7519          | 0.7519                | 1.0000                | 0.8584                 | 0.4292                |
| Original BERT (untrained head)  | 0.7013          | 0.7517                | 0.9000                | 0.8192                 | 0.4802                |
| Fine-tuned BERT (3 epochs)      | 0.8752          | 0.8991                | 0.9395                | 0.9189                 | 0.8245                |

The macro-F1 tells the real story. Because the dataset is imbalanced, the binary F1 for the majority baseline still looks deceptively high. In contrast, macro-F1 shows that the untrained BERT is only slightly better than a trivial majority-class predictor, whereas fine-tuning raises macro-F1 from 0.4802 to 0.8245, an improvement of 0.3442. Compared with the majority baseline, the macro-F1 gain is even larger at 0.3953. The most important change is in fake-news recall: the untrained BERT detects only 9.9% of fake items, while the fine-tuned model raises fake recall to 68.0%. This makes the difference between a model that mostly follows the majority class and one that can detect a substantial share of suspicious articles.

## 4.2 NLI Verifier Evaluation

The final verifier was fine-tuned from **MoritzLaurer/deberta-v3-base-mnli-fever-anli** on the custom balanced news NLI dataset described in Section 2.2. Evaluation was performed on a held-out test set of 4,500 samples. Earlier heuristic matching logic was useful during prototype development, but the final deployed verifier is the fine-tuned NLI model reported below. On this test set, the fine-tuned verifier reaches accuracy 0.8644, macro precision 0.8649, macro recall 0.8644, and macro-F1 0.8643. At the class level, contradiction, neutral, and entailment F1 are 0.8446, 0.8410, and 0.9072 respectively. These results show that entailment is the strongest class, while contradiction and neutral remain slightly harder. This is expected in news verification because retrieved evidence often contains partial overlap or incomplete context. Even so, the macro F1 of 0.8643 indicates that the verifier is robust enough to serve as the semantic core of the final system. Table 7 shows the effect of domain-specific fine-tuning more directly by comparing the original pretrained NLI checkpoint with the final news-tuned verifier on the same balanced 4,500-sample test set.

Table 7: Original versus fine-tuned NLI verifier on `real_nli_test.jsonl`

| Metric            | Original<br>DeBERTa-v3 | Fine-tuned (News<br>NLI) | $\Delta$ Delta |
|-------------------|------------------------|--------------------------|----------------|
| Accuracy          | 0.6816                 | 0.8644                   | +0.1829        |
| Precision (macro) | 0.7200                 | 0.8649                   | +0.1450        |
| Recall (macro)    | 0.6816                 | 0.8644                   | +0.1829        |
| F1 (macro)        | 0.6807                 | 0.8643                   | +0.1836        |

Table 8: Per-class F1 improvement after news-specific NLI fine-tuning

| Class         | Original | Fine-tuned | $\Delta$ Delta |
|---------------|----------|------------|----------------|
| Contradiction | 0.6474   | 0.8446     | +0.1972        |
| Neutral       | 0.6641   | 0.8410     | +0.1769        |
| Entailment    | 0.7307   | 0.9072     | +0.1765        |

The fine-tuning improvement is substantial: macro-F1 increases by 18.36 percentage points. The most critical gain is contradiction recall, which rises from 52.13% to 81.87%. This matters operationally because missed contradictions would cause the fact-checker to overlook refuting evidence and produce overly optimistic support labels. The original checkpoint also shows strong neutral-to-entailment confusion, especially on evidence with topical but not fully supportive overlap. In the original model, contradiction recall is only 0.5213 and contradiction F1 is 0.6474, while neutral precision is only 0.5673. Fine-tuning on 30,000 news-specific NLI pairs substantially reduces those weaknesses and improves all three classes consistently.

### 4.3 Model Comparison Summary

Across both learned components, the results show that domain-specific fine-tuning consistently improves performance over generic pretrained baselines. For the NLI verifier, macro-F1 improves by 18.4 percentage points after news-specific training. For the baseline fake/real classifier, macro-F1 improves by 34.4 percentage points over the untrained BERT head. This comparison supports the main design choice of the project: pretrained language models provide a strong initialization, but task-specific fine-tuning on domain-relevant news data is what makes the system reliable enough for practical evidence-based verification.

### 4.4 End-to-end Verification Case Study

A complete evaluation should also include one worked example showing how the system behaves on a real article. Table 9 gives a suitable format for documenting sentence-level evidence, predicted labels, and the type of improvement expected from the NLI verifier.

This kind of case study is important because it demonstrates not only whether the final verdict is plausible, but also whether the evidence trail shown to the user is understandable and trustworthy. In particular, the NLI verifier is more reliable than simple overlap matching when numeric values or partially related phrases appear in the evidence but do not actually contradict the claim.

Table 9: Representative end-to-end case-study structure

| Sentence                                                                         | Top Evidence   | Target Label    | Comment                                                                                      |
|----------------------------------------------------------------------------------|----------------|-----------------|----------------------------------------------------------------------------------------------|
| Brockman disclosed financial ties and a nearly \$30B stake.                      | Reuters        | Supported       | Strong evidence match for the financial-interest statement.                                  |
| Musk filed a lawsuit over the for-profit versus nonprofit issue.                 | Economic Times | Supported       | Retrieved article directly addresses the litigation context.                                 |
| Brockman’s independence may be compromised by startup investments.               | Rappler        | Supported       | Evidence is related and provides supporting discussion rather than exact lexical overlap.    |
| Stakes in startups and Altman’s family fund raise conflict-of-interest concerns. | Economic Times | Supported       | Useful example of multi-entity matching across one sentence.                                 |
| The trial and ChatGPT launch happened in 2022.                                   | MarketScreener | NEI → Supported | Heuristic matching can overreact to unrelated numbers; NLI is intended to reduce this error. |

#### 4.5 Error Analysis and Discussion

Even after the NLI upgrade, several error sources remain important:

- **Retrieval errors:** GNews may return related but still insufficient evidence.
- **Snippet limitation:** the returned content can be truncated, which weakens premise quality for NLI.
- **NLI ambiguity:** long or multi-fact sentences can confuse the verifier because one sentence may contain both supported and unsupported subclaims.
- **Label-mapping noise:** stance datasets do not perfectly match factual entailment labels.
- **Aggregation sensitivity:** a single wrongly refuted sentence may affect the article-level verdict disproportionately.

Including this discussion makes the report more realistic and shows that the team understands the limitations of evidence-based verification in practical deployment settings.

### 5 Limitations and Future Work

The current system is more transparent than a pure article-level classifier, but it still depends heavily on retrieval quality and dataset fit. If the retrieved news snippets are incomplete or weakly related, even a strong NLI model cannot produce reliable verification labels. In addition, the baseline fake/real data are title-heavy, so they are better suited to screening than to deep factual reasoning. Future work should focus on four areas. First, the team should expand the NLI training set with more news-specific claim-evidence pairs and a cleaner manual validation split. Second, the retrieval stage can be improved with better sentence selection, reranking, and source filtering. Third, the

aggregation rule can be calibrated with confidence weighting rather than simple counting. Fourth, the system can be extended to support multi-hop evidence and longer article-body verification beyond sentence-level snippets.

## 6 Conclusion

The final system demonstrates a hybrid approach to news verification. The BERT classifier provides a fast article-level fake/real prior, while sentence-level GNews retrieval and the fine-tuned NLI verifier provide evidence-based semantic verification. Sentiment analysis, LDA topic modelling, and the chat interface improve interpretability and usability.

Compared with a pure fake-news classifier, this design is more transparent because it shows which sentences are supported, refuted, or still unverified and which external evidence was used. At the same time, the system remains limited by retrieval quality, snippet completeness, label-mapping noise, and the final accuracy of the NLI verifier. Even with those limitations, the project shows why evidence-based verification is a more convincing direction than relying only on article-level style classification.

## 7 References

### 7.1 Libraries and Frameworks

1. **Streamlit**. Used for the interactive application interface. Official documentation: Streamlit documentation. Chat component reference: Streamlit chat API reference. Public source code: streamlit/streamlit.
2. **PyTorch**. Used as the deep learning runtime for BERT and DeBERTa training and inference. Official documentation: PyTorch documentation. Public source code: pytorch/pytorch.
3. **Hugging Face Transformers and Datasets**. Used for model loading, tokenization, fine-tuning, and dataset preparation. Official documentation: Transformers documentation and Datasets documentation. Public source code: huggingface/transformers and huggingface/datasets.
4. **scikit-learn**. Used for train-test splitting, evaluation metrics, `CountVectorizer`, and LDA topic modelling. Official user guide: scikit-learn user guide. LDA reference: LatentDirichletAllocation documentation. Public source code: scikit-learn/scikit-learn.
5. **pandas, NumPy, and Requests**. Used for CSV processing, numerical operations, and API calls. Official documentation: pandas, NumPy, and Requests.
6. **NLTK / VADER**. Used as a fallback sentiment analyser when the transformer sentiment model is unavailable. Official documentation: NLTK documentation. VADER reference: VADER repository.
7. **BERTopic**. Supported as an alternative topic-modelling method when the environment supports it. Official documentation: BERTopic documentation. Public source code: MaartenGr/BERTopic.

### 7.2 Models, APIs, and Data Sources

1. **bert-base-uncased**. Pretrained encoder used for the baseline fake/real classifier. Model card: Hugging Face model card.
2. **DeBERTa / DeBERTa-v3**. Backbone used for semantic verification through NLI fine-tuning. Model resources: DeBERTa-v3 base model card.
3. **MoritzLaurer/deberta-v3-base-mnli-fever-anli**. Practical starting checkpoint for NLI-

based verification. Model card: Hugging Face model card.

4. **News-related NLI or stance-verification data.** Used to build claim-evidence-label training pairs for the verifier. One public reference is the Fake News Challenge Stage 1 dataset: FNC-1 repository. The report should note that stance labels require mapping and manual inspection before use as NLI labels.
5. **GNews API.** Live evidence source used at inference time for sentence-level retrieval. Official documentation: GNews API documentation.
6. **cardiffnlp/twitter-roberta-base-sentiment-latest.** Sentiment model used as an interpretability feature. Model card: Hugging Face model card.
7. **Flan-T5.** Backbone used for the auxiliary claim extraction module. A practical reference checkpoint is: google/flan-t5-base model card.
8. **FakeNewsNet-style GossipCop and PolitiFact data.** Local fake/real datasets used for the baseline classifier and topic analysis. Public repository reference: KaiDMML/FakeNewsNet.

## 8 Individual Contributions

Table 10: Planned contribution summary for the final report

| Team Member  | Main Responsibility                                                                           | Contribution Details                                                                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sun Yuchen   | Dataset preparation, base-line BERT training, evaluation, and topic analysis                  | Prepared the fake/real datasets, organized the baseline training workflow, documented classifier evaluation, and supported LDA-based dataset exploration.                                                  |
| Zhao Jiahui  | GNews retrieval, sentence-level pipeline, Streamlit app, chat interaction, and topic analysis | Implemented retrieval logic, sentence-level verification flow, evidence display, the user-facing interface for verification and chat-style interaction, and the topic analysis page with BERTopic support. |
| Zhang Yuxuan | NLI dataset construction, DeBERTa-NLI fine-tuning, sentiment module, and verifier evaluation  | Prepared claim-evidence training pairs, fine-tuned the NLI verifier, integrated sentiment analysis, and designed the verifier comparison metrics.                                                          |
| Shared       | Report writing, demo preparation, and error analysis                                          | All team members contributed to report revision, final presentation materials, discussion of limitations, and interpretation of system results.                                                            |

## 9 GenAI / LLM Declaration

Generative AI tools were used to assist with debugging, report structuring, explanation refinement, and code review. The team manually reviewed, modified, and validated the generated suggestions before including them in the project. No confidential or private user data was submitted. The final system design, implementation decisions, experiments, and conclusions remain the responsibility of the project team.